OV VO

Getting OVVO onto a real account.

Everything I need from you, in the order you need it. Cloudflare first, because it gives you the address Firebase has to be told about. Firebase second, because it needs that address. Nothing here costs money and nothing needs a card.

11
PARTS

30 min
OF YOUR TIME

2
FREE ACCOUNTS

1	Make a Cloudflare account and claim your workers.dev address. First, because everything else asks for it.
2	Copy the account ID and make one scoped token, the only secret in this guide.
3	Make a Firebase project, so a traveller can sign in with an email or a Google account.
4	Register the web app and copy six public values, which is all I need from Firebase.
5	Turn on email and password sign-in.
6	Turn on Google sign-in and publish the consent screen, so strangers are allowed in.
7	Allow the address from part 1 to start a sign-in.
8	Send me the values in one paste, and put the one secret where secrets go.
9	Check it works before I build on it.
10	The failures this setup actually produces, and what fixes each one.
11	What happens after that, and what I promise about your year.

OVVO · setup guide · written for the steerer, not for the code

PART 1

Claim your address on Cloudflare

This is first because two later parts ask for your workers.dev address, and you cannot type an address into Firebase before it exists. Ten minutes here and every later part has its value ready. No card, and nothing to pay.

1.1 Make the account

WHERE https://dash.cloudflare.com/sign-up — an email and a password is all it asks for.

Confirm the email they send you, then sign in to the dashboard.

1.2 Find your subdomain

WHERE In the left sidebar, click **Workers & Pages**. On the right of that page, in a panel headed with your account name, there is a line called **Your subdomain** with a **Change** link beside it.

If you have never used Cloudflare, it will say you need to choose one. Click **Change**, type a name you are happy to see in public — **yourname** is good — and save it.

CAREFUL A subdomain can be changed later, but that changes the address, and the address is what Firebase is told to trust. Pick it once, now, and leave it.

1.3 Write down your address

Our app on Cloudflare is called **ovvo**, and a Cloudflare app's public address is its name in front of your subdomain. So yours will be:

```
https://ovvo.your-subdomain.workers.dev
```

If you choose the subdomain ovvo, the address becomes https://ovvo.ovvo.workers.dev. That is correct, just ugly. Choosing your own name avoids it.

COPY THIS Write the whole address down where you can find it in part 7. Nothing needs to be deployed for the address to be yours.

1.4 Copy the account ID

WHERE The same panel on **Workers & Pages** shows **Account ID** with a copy icon: a long string of letters and numbers.

COPY THIS The account ID is **not** a secret. It goes in the paste in part 8.

PART 2

Make the deploy token

One token, three permissions, and the only secret in the whole guide. It lets me put the site and the database up without you having to run anything.

2.1 Find the token page

WHERE — Click your profile icon, top right, then **Profile**, then **API tokens** in the left menu. On the newer dashboard it is **Manage Account → Account API tokens**.

Click **Create Token**, then at the bottom of the template list click **Create Custom Token**, and name it **OVVO deploy**.

2.2 Give it exactly three permissions

Under **Permissions**, add these three rows one at a time, each with **Account** as the scope and your account chosen under **Include**:

Workers Scripts → Edit, then **D1 → Edit**, then **Account Settings → Read**.

Those three cover the whole of what I need: the site itself, the database that will hold the year, and the account details that tie them together. You do not run any commands at any point, and you never need to open a terminal.

CAREFUL — The ready-made "Edit Cloudflare Workers" template does **not** include D1. If you pick it, the deploy fails later with a permission error that looks unrelated, so build the custom token instead.

2.3 Copy it once, and put it where secrets go

Click **Continue to summary**, then **Create Token**. The value is shown **once** and starts with cfut_.

Paste it into this project under **Settings → Secrets**, as a secret named CLOUDFLARE_API_TOKEN. Add CLOUDFLARE_ACCOUNT_ID there too, with the account ID from part 1.

CAREFUL — Never paste the token into a chat message, a document, a screenshot or a code file. If it lands somewhere it should not, delete it in the token list and make a new one; nothing is lost by that.

PART 3

Make the Firebase project

Firebase is Google's sign-in service. The free plan is permanent, needs no card, and covers far more travellers than this site will ever have. One project serves development, the preview and the live site.

3.1 Sign in to the Firebase console

WHERE — https://console.firebase.google.com — sign in with the Google account you want to own OVVO. Not a work or school account: those can be closed by an administrator and take the project with them.

3.2 Create the project

Click **Create a project**, or **Add project** at the top.

Type a name. **OVVO** is fine; the name is only what you see in the console.

COPY THIS — Click the pencil beside the auto-generated **project ID** and set it to something clean, for example `ovvo-app`. It appears in a public web address later and **cannot be changed afterwards**.

3.3 Turn Analytics off

The next screen offers Google Analytics. It is optional and we do not use it, so switch it **off**.

Accept the terms if asked, click **Create project**, wait, then **Continue**.

CAREFUL — Do not click anything offering to "upgrade" the project. The paid platform changes the free limits from generous to three thousand people a day.

PART 4

Register the web app and copy six values

The only thing I need out of Firebase. All six are public by design: they name the project, they do not unlock it, and Firebase says so in its own documentation.

4.1 Add a web app

On the project overview page, click the **web** icon (a small screen with a bracket, labelled </>).

Give it a nickname, for example **OVVO web**. Leave the Firebase Hosting checkbox **unticked**: we are not using their hosting.

Click **Register app**.

4.2 Copy the config block

The next screen shows a block beginning `const firebaseConfig = {`. Copy it into a note, or leave the tab open.

COPY THIS — The six values I need: `apiKey`, `authDomain`, `projectId`, `storageBucket`, `messagingSenderId` and `appId`.

4.3 If you lose it, it is always here

WHERE — Gear icon → **Project settings** → **General** tab → scroll to **Your apps** → your web app → **SDK setup and configuration** → **Config**. The same block, at any time, forever.

4.4 Two of the values do double duty

projectId is also on that General tab, in the top pane, labelled **Project ID**.

authDomain is normally your-project-id.firebaseapp.com. Keep it exactly as Firebase gave it. Changing it to your own address breaks Google sign-in until extra Google Cloud setup is done, and we do not need that trade.

PART 5

Turn on email and password

The sign-in that works with nothing but an address and a password, and the one that has to work on a new phone after a forgotten password.

5.1 Open Authentication

WHERE Left sidebar → **Security** → **Authentication**. Some consoles show **Build** → **Authentication**. It is the item called Authentication either way.

5.2 Enable the provider

Open the **Sign-in method** tab. In the provider list click **Email/Password**, turn its toggle on, then click **Save**.

Leave **Email link (passwordless sign-in)** off for now.

5.3 Do not add anything else

Ignore phone, anonymous and the rest. Every provider you switch on is another way in and another thing to keep right.

Firebase sends its own password reset and verification emails on the free plan, so no email service and no domain are needed for that.

PART 6

Turn on Google sign-in

One tap with the Google account already on the phone. This part has the most hidden steps, so it has the most detail.

6.1 Enable the provider

Still in **Authentication → Sign-in method**, click **Google** and turn its toggle on.

It asks for a **project public-facing name** (type **OVVO**) and a **support email** (pick yours).

Click **Save**.

COPY THIS — You do **not** need to create a Google OAuth client. Firebase keeps its own for you, which is why part 4 says to leave `authDomain` alone.

6.2 Set up the consent screen once

WHERE — https://console.cloud.google.com — the same project is there under the name you gave it. Open the menu and choose **Google Auth platform**. If it says it is not configured yet, click **Get Started**.

6.3 Fill in the four things it asks for

App name: OVVO. **User support email:** your address. Click **Next**.

Audience: choose **External**. That sounds public, and it is: anyone with a Google account, not only people inside an organisation. Click **Next**.

Contact information: your address again. Click **Next**.

Tick the box agreeing to the user data policy, click **Continue**, then **Create**.

6.4 Publish it

On the **Audience** page you will see **Publishing status: Testing**. Click **Publish app** and confirm.

While it says Testing, only a hundred named test accounts can sign in and their sign-ins expire after seven days.

CAREFUL — Publishing with only the basic profile and email scopes does **not** need Google's review, so it works immediately. Only the name and logo on the consent screen may lag, which is cosmetic.

PART 7

Allow your address to start a sign-in

Firebase refuses to begin a sign-in from an address it does not know, even a perfectly good one. The address you need is the one you wrote down in part 1.3.

7.1 Find the list

WHERE **Authentication → Settings** tab → scroll to **Authorized domains**. It is the Settings tab, not the Sign-in method tab; older guides get this wrong.

7.2 Add your address

Click **Add domain** and type the host name only: no https:// in front, no slash at the end, no path.

COPY THIS From part 1.3, `ovvo.your-subdomain.workers.dev` goes in the box. If your address is `https://ovvo.ovvo.workers.dev`, you type `ovvo.ovvo.workers.dev`.

If the address I deploy to ever turns out to differ from this one, I tell you, and you change this single line. Nothing else in Firebase has to change.

7.3 Add localhost

Click **Add domain** again and add localhost, on its own.

Projects made after April 2025 no longer include it, and without it nothing works on your own machine while I am building.

7.4 Leave the two Firebase addresses alone

You will see your-project-id.firebaseapp.com and your-project-id.web.app already listed. Do not remove them: the sign-in window travels through them even when it opens from our address.

PART 8

What to send me

One message, one paste block. Your address comes first, because it is what I need to start building. Everything below is safe to send.

Copy this, fill it in, send it

Leave a line blank if you have not done that part yet. I will tell you what is missing rather than guessing.

```
workers.dev address (part 1.3):

Cloudflare account ID (part 1.4):

Cloudflare token saved in Secrets?  yes / no

Firebase project ID:

Firebase apiKey:

Firebase authDomain:

Firebase storageBucket:

Firebase messagingSenderId:

Firebase appId:

Email + password turned on?         yes / no

Google turned on?                   yes / no

Consent screen published?           yes / no

Address added to Firebase?          yes / no
```

CAREFUL The Cloudflare token does not go in this block. It lives only in Settings → Secrets, and I can see that it is there without ever seeing its value.

PART 9

Check it before I build on it

Five minutes now saves a confusing afternoon later. Do these once the paste is sent; I will run the same checks from my side.

- [ ] On Cloudflare, **Workers & Pages** shows your subdomain, and it matches the address you sent.
- [ ] The token exists in the token list and is called OVVO deploy.
- [ ] Under this project's Settings → Secrets, both names appear with a value set.
- [ ] In **Authentication → Sign-in method**, Email/Password and Google both show as Enabled.
- [ ] In **Authentication → Settings → Authorized domains**, your address and localhost are both listed.
- [ ] In the Google console, **Google Auth platform → Audience** says **In production**, not Testing.
- [ ] The address you typed into Firebase is character for character the one on the first line of your paste.
- [ ] In Firebase, **Project settings → General** still shows your web app and the config block opens.

Then, once I have the first version up, the real test is four actions on your own phone: sign in with an email and password, sign out, sign in with Google, and confirm the year on screen is still the same year.

PART 10

If something goes wrong

These are the failures this setup actually produces, what they look like, and what fixes them. None of them is a broken build.

> **auth/unauthorized-domain**
> The sign-in refuses to start and names the address. Part 7 was skipped, or the address was typed with https:// in front of it. Add the bare host name.

> **Error 400: origin_mismatch**
> The Google window opens and then blocks. That one belongs to a Google OAuth client, which we deliberately do not use. It appears only if authDomain was changed in the Firebase config; put back the firebaseapp.com value from part 4.

> **The popup opens and closes, or the browser blocks it**
> Google sign-in from a site that is not hosted by Firebase has to use the popup, and some browsers are strict about it. A second click usually works; if it never does, I add a small proxy in the worker so the redirect route works too.

> **A reset email never arrives**
> On the free plan the mail comes from a firebaseapp.com address and spam filters are wary of it. Check spam before assuming the setup is wrong. If it becomes a habit, the fix is a mail domain, the one thing in this plan that costs money.

> **Your workers.dev page errors for the first minute**
> Cloudflare's own documentation says a brand-new workers.dev address can throw 523 errors for about a minute after the first deploy. Wait and reload; it is not a bad deploy.

PART 11

After that

So you know what you are waiting for, and what I will not do with any of it.

What I build next. The site and a small API on Cloudflare, a database table for the year, the dated tally underneath the Wrapped, and an account card in Settings rather than in the top row. The whole app keeps working signed out.

What the database holds. Dated events: a journey planned, a board opened, a stop hearted, a theme worn. They belong to an account and nobody else can read them, because every query is filtered by the signed-in account on the server.

What it never holds. Your name, your email, your location, your password. Firebase keeps the account; the database keeps the year and only the identifier Firebase gives it.

The controls that go with it. One to forget a year, one to delete everything the account holds, and a plain sentence in Settings saying what leaves the device.

The honest limit. A year can only be counted from the day the tally starts. The first December will be a short year, and the card will say so rather than dressing up three journeys as a life.